CAMPUSBITE

College Food Ordering & Smart Canteen Management

COMPLETE PRODUCT REQUIREMENTS DOCUMENT (PRD) + TECHNICAL ARCHITECTURE + LOCAL MVP DELIVERY PLAN

Student App | Kitchen App | Super Admin Dashboard

React Native + Expo • Python FastAPI • PostgreSQL • Redis • Docker

Version 1.0 | 18 October 2026

Scope: local-first working MVP, designed for multi-college expansion. This document explains every major functional module, flows, UI behavior, database design, APIs, security, test cases, delivery phases, and rollout decisions.

Design direction: premium, fast, Blinkit-inspired interaction quality using original branding, layouts, assets, and components. Not a pixel-for-pixel reproduction.

01 / Executive summary

CampusBite serves three audiences: students ordering food, college kitchen owners fulfilling it, and an operator administering colleges, vendors, finances, and support. The initial operational model is counter pickup within an approved college. On-campus delivery is a later module, not a launch dependency.

- Three interfaces: Student React Native app, Kitchen React Native app, and desktop-friendly Admin panel.
- One modular Python FastAPI service for the MVP, backed by PostgreSQL and Redis.
- Reliable order lifecycle: checkout -> payment/cash confirmation -> kitchen acceptance -> preparation -> ready -> one-time pickup verification -> completion.
- Local developer setup using Docker Compose, deterministic demo seed data, and test/mocked external integrations.
- Success criterion: a student can order from an approved kitchen, staff can fulfill it, and the system retains correct payment, stock, and order history.

Decisions and boundaries

Decision	MVP choice	Reason
Database	PostgreSQL	Strong relational constraints, transactions, JSONB, optional PostGIS later.
Cache	Redis	Transient caching, rate limits, Pub/Sub, jobs; never sole durable order store.
Ordering	One kitchen per order	Predictable prep, inventory, payment and pickup.
Fulfillment	Counter pickup by QR/OTP	No rider logistics during pilot.
Payments	Mock/test mode; optional cash	No production settlement dependency during local build.
Identity	Local fake OTP; pluggable live provider	Fast local tests without SMS spend.
UI direction	Blinkit-inspired, distinct CampusBite brand	Comparable speed and polish without cloning protected assets.

02 / Users, roles and access boundaries

Role	Can do	Cannot do
Student	Browse permitted college kitchens; maintain profile; place/pay/track orders; view receipts; request support.	Manage menus; inspect other students; finalize order statuses.
Kitchen owner	Manage own kitchen, staff, schedule, menu, stock, incoming orders and reports.	Manage unrelated kitchens or colleges.
Kitchen cashier	View assigned order queue; accept, ready, verify pickup if allowed.	Change payout settings or global configurations.
Kitchen cook	View preparation queue; mark preparing/ready.	See financial admin details or alter price.
College admin (optional)	View college-local roster, kitchen approvals and selected operational reports.	View other colleges or platform-wide finances.
Platform super admin	Manage colleges, kitchens, moderation, payments, support, flags, reporting.	Bypass audited financial actions without tracking.

All protected reads and writes must enforce scope on the server, using authenticated user ID, role and college/kitchen membership. Never trust a college_id or kitchen_id merely because the mobile app sent it.

03 / Detailed student application: onboarding and discovery

Splash, introductory onboarding and first-run permissions [MVP]

Purpose: Communicate fast campus pickup, kitchen variety, and order tracking.

User flow: Brand splash -> 2-3 concise slides -> Get Started -> login. Ask location only when discovering nearby colleges, not on initial splash.

Business rules: Remember completed onboarding; honor reduced-motion preference; clear privacy messaging.

Exceptions and acceptance: No network still displays local onboarding; denied location does not block manual college search.

Phone authentication and user profile [MVP]

Purpose: Create a stable identity linked to college and order history.

User flow: Enter mobile -> request OTP -> verify -> enter name and optionally email, roll number, class/year.

Business rules: Throttle OTP requests; short-lived OTP; server-issued access/refresh tokens; sensitive data minimized.

Exceptions and acceptance: Wrong/expired OTP shows retry; rate-limited states; local mode provides controlled deterministic test OTP.

Location-aware college discovery [MVP]

Purpose: Suggest likely campus without silently assigning one.

User flow: User grants location -> display colleges sorted by distance -> search and manually choose -> confirm campus.

Business rules: Location is optional; campuses have verified addresses and coordinates; college eligibility policy may require verified email/ID.

Exceptions and acceptance: Denied permission, poor GPS and outside-campus users can search/select manually; location not trusted for access control.

Home and campus context [MVP]

Purpose: Fast glanceable ordering hub optimized for repeat visits.

User flow: Top campus switcher -> search -> promo banner -> food categories -> open kitchens -> popular meals -> sticky cart when nonempty.

Business rules: Show open status, food prices, dietary labels, approximate preparation time and sold-out status from API.

Exceptions and acceptance: Campus switch prompts cart clear if cross-college; offline/retry/empty states are deliberate.

Kitchen directory, search and filters [MVP]

Purpose: Allow students to choose an approved food outlet quickly.

User flow: Browse approved open/closed kitchens; filter by vegetarian, category, availability, price; sort by prep estimate or relevance.

Business rules: Closed kitchens remain discoverable but cannot accept immediate orders; ratings only from eligible completed orders.

Exceptions and acceptance: No results -> clear filters; temporarily closed -> optional scheduling if supported.

Kitchen page and detailed food menu [MVP]

Purpose: Present complete and accurate menu options.

User flow: Open kitchen -> image/header/hours -> category tabs -> product cards -> detail sheet -> variant/add-on selection.

Business rules: Each item has SKU, price in paise, status, option groups, max per order, taxes if applicable.

Exceptions and acceptance: Sold-out options disabled, missing images use placeholder; price changes trigger cart revalidation.

Food search and saved favorites [MVP]

Purpose: Find specific dishes with minimal friction.

User flow: Search dish across selected college -> results grouped by kitchen; toggle favorite and revisit.

Business rules: Search index scoped by college; never show private/unapproved kitchens.

Exceptions and acceptance: Handle spelling variations, zero matches, unavailable food, and network delay.

04 / Detailed student application: checkout to pickup

Cart, quantity stepper and price summary [MVP]

Purpose: Create an auditable basket with transparent total.

User flow: Tap Add -> animated quantity stepper -> sticky View Cart -> edit quantities/instructions -> price breakdown.

Business rules: MVP enforces one kitchen per cart; store cart locally but verify server-side stock, pricing, taxes and offers at checkout.

Exceptions and acceptance: If student tries another kitchen, ask to replace cart; reject negative quantity; stale price prompts confirmation.

Scheduling and pickup method [MVP]

Purpose: Reduce campus queueing and kitchen overload.

User flow: At checkout choose ASAP or supported pickup timeslot; display estimated readiness window and collection counter.

Business rules: Timeslots limited by opening hours, capacity, lead time and item availability. Immediate pickup only is acceptable for earliest pilot.

Exceptions and acceptance: Kitchen unexpectedly closes -> cannot confirm; slot fills -> offer next available; never promise guaranteed times.

Payment and checkout [MVP]

Purpose: Collect verified payment or cash order commitment.

User flow: Confirm items/address-free pickup context -> choose test UPI/card or pay at counter -> create pending order -> verify server-side payment -> acknowledge.

Business rules: Amount calculated on backend; payment state separate from fulfillment state; idempotency prevents duplicate order creation; gateway webhook is canonical confirmation when live.

Exceptions and acceptance: Timeout -> pending state and reconciliation, not false success; failed payment -> retry safely; refund handled as separate state.

Order confirmation and real-time tracking [MVP]

Purpose: Remove uncertainty after purchase.

User flow: Show order number, kitchen, items, ETA -> status timeline updated via WebSocket -> notification on accepted, delayed or ready.

Business rules: Events contain order version; client reconnect fetches canonical latest order via REST; ETA is estimate not guarantee.

Exceptions and acceptance: Kitchen rejects -> cancellation/refund path; socket disconnect -> polling fallback; duplicate events do not duplicate UI.

Pickup code / QR claim [MVP]

Purpose: Prove the right student has collected the meal.

User flow: Once ready, display short-lived QR plus accessible numeric pickup code -> staff scans or enters -> successful verification completes order.

Business rules: Generate unpredictable one-time token; store only hash server-side; expire/rotate; atomic consume after checking order ownership and READY status.

Exceptions and acceptance: Bad/used/expired token rejected; staff can request supervised fallback; QR screenshot alone should not bypass expiry checks.

Order history, reorder and help [MVP]

Purpose: Support everyday repeat usage and disputes.

User flow: View past orders -> receipt -> reorder available items -> contact support for delayed, missing or wrong order.

Business rules: Reorder rechecks current menu prices and availability; support tickets reference permitted order ID.

Exceptions and acceptance: Items deleted or sold out are omitted with explanation; refunds never assumed until verified.

Ratings, coupons and loyalty [Phase 2]

Purpose: Retention and trust after basics work.

User flow: Rate completed order; apply eligible coupon; earn loyalty points based on finalized rules.

Business rules: Reviews only for completed purchases; server validates code, minimum basket and usage limits.

Exceptions and acceptance: Refunded order loyalty reversed per policy; review abuse moderation.

Group orders and recurring mess plans [Phase 2]

Purpose: Broaden utility after operational baseline.

User flow: Group host shares order session; friends add/pay; kitchen sees clear consolidated fulfillments. Mess subscribers manage meal calendar and skips.

Business rules: Separate ledger entries and kitchen capacity enforcement are mandatory; subscription changes need cutoff rules.

Exceptions and acceptance: Partial failure, friend no-pay, skipped meals, holidays and refunds need explicit policies.

05 / Detailed kitchen application

Kitchen registration, verification and approval [MVP]

Purpose: Protect college identity and platform quality.

User flow: Owner signs up -> chooses college -> submits business/kitchen information, counter location, bank details only if needed -> awaits approval.

Business rules: Kitchen remains DRAFT/PENDING until admin approves; check required food safety and college permissions; document access restricted.

Exceptions and acceptance: Duplicates flagged; rejected owners receive actionable reason; license expiries prompt review.

Kitchen home, operating hours and busy mode [MVP]

Purpose: Make readiness visible and control demand.

User flow: Owner opens dashboard -> toggles accepting orders -> sets weekly hours, breaks and holiday exceptions -> sets estimated prep delay.

Business rules: Closing kitchen blocks new orders, not the duty to finish/refund accepted orders; explicit timezone Asia/Kolkata.

Exceptions and acceptance: Unexpected outage emits notices and operator escalation; busy mode can extend lead times or pause incoming demand.

Menu, options, categories and assets [MVP]

Purpose: Let owner maintain menu without technical help.

User flow: Add category -> product image/name/description/base price -> veg marker -> sizes/variants/add-ons -> publish.

Business rules: Validation for currency, max image sizes, option constraints; unapproved media storage kept private; server sanitizes text.

Exceptions and acceptance: Draft edits do not corrupt existing order snapshots; missing images use default.

Stock and temporary sold-out [MVP]

Purpose: Prevent sales of unavailable food.

User flow: Set stock as unlimited/limited; mark item/variant sold out; automatically decrement reserved stock when order committed.

Business rules: Concurrency-safe row-level updates; rejected or expired reservations restore stock according to policy.

Exceptions and acceptance: Two orders competing for last portion: exactly one should succeed; manual stock changes are audited.

Live order queue and audible notifications [MVP]

Purpose: Enable quick reaction to campus peaks.

User flow: Tabs New / Accepted / Preparing / Ready / Completed -> sound/vibration for new -> accept/reject -> cook begins -> mark ready.

Business rules: Status permissions by role, monotonic versioning, no duplicate fulfillment, reject reason required, configurable acknowledgement timeout.

Exceptions and acceptance: Offline queue recovers by API fetch; staff session expires securely; reject after payment triggers refund path.

Order preparation, timing and pickup validation [MVP]

Purpose: Coordinate kitchen execution.

User flow: Owner sets 10/15/20-minute estimate -> updates progress -> staff verifies QR or code at collection counter.

Business rules: Do not mark completed until token verified (or audited exception); ETA may change with visible notice.

Exceptions and acceptance: Late prep alerts and exceptions can be surfaced to student and admin.

Kitchen staff and role management [MVP]

Purpose: Delegate routine tasks safely.

User flow: Owner invites staff -> sets cashier/cook/manager role -> removes access on departure.

Business rules: Per-kitchen role checks; ownership transfers audited; staff sees minimum necessary personal information.

Exceptions and acceptance: Suspended staff instantly loses token privileges; no cross-kitchen visibility.

Kitchen sales, settlement and reports [MVP]

Purpose: Show transparent performance.

User flow: Dashboard offers gross sales, completed order count, cancellations, payouts, discounts, refunds, peak hours, top items.

Business rules: Distinguish gross merchandise value, platform fee, gateway fee, tax and net payout. Reports derive from immutable transactional data.

Exceptions and acceptance: Cancelled/refunded items not counted as completed sales; report export from authorized kitchen only.

06 / Detailed admin and business operations

Campus directory

Create/verify campuses with location, address, administrative contacts, code, active/inactive state, kitchen eligibility and schedules. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Kitchen moderation

Review identity, approvals and documents; suspend vendor, record notes and history, show policy reasons. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Student management

Search users, resolve account issues, flag abuse, manage college association with an audit trail. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Live operations

Filter all orders by college, kitchen, time, status and payment health; identify delayed or rejected orders. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Financial controls

Trace each payment, refunds and adjustments. Export reconciliations and ensure test/prod separation. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Offer engine

Scope discounts to college, kitchen, time window, payment method or student segment; limit usage. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Support tickets

Track issue category, assigned operator, status, SLA, notes, order relation and resolution. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Insights dashboard

Aggregate daily active students, conversion, orders per kitchen, preparation time, cancellations, repeat buyers and net revenue. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Configuration

Manage category taxonomy, rollout feature flags, notification templates, maintenance windows and policy pages. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

Audit and privacy

Log high-impact changes, apply least-privilege views and configurable retention for sensitive student data. Acceptance: role-scoped results, pagination, error/empty states, audit record for mutating action, and export only when authorized.

07 / UX design and animation playbook

Visual direction: warm campus yellow (#F8C81C), deep green (#166534), white surfaces, charcoal text, generous 12-16 px corners, photo-led menus, consistent 8-point spacing, large readable CTAs, and highly responsive scrolling. These are starting tokens rather than fixed brand requirements.

Pattern	Detailed specification
Hero and header	Selected campus is always evident; compact address/campus dropdown; search prominent; clear open/closed badges.
Food tiles	Square/4:3 images, two-line name, price, availability, accessible Add button; avoid excessive decorative chrome.
Sticky cart	Visible only with nonempty cart; slide/spring animation; total and item count; bottom safe-area handling.
Product detail	Accessible sheet with name, food image, variants, add-ons, quantity and clear final price.
Microinteractions	Press scale 0.98, 150-250 ms feedback, stepper spring, restrained staggered entry, shimmer for load.
Order state	Progress timeline, ETA disclaimer, animated new/ready signals; never convey status by color alone.
Accessibility	Minimum 44x44 logical touch target where practical, contrast, semantic labels, text resize, reduced-motion mode.
Performance	Virtualize long menus, compress images, paginate queries, prefetch selective details, avoid heavyweight animations on JS thread.

08 / Screen inventory and navigation

Application	Navigation / screens
Student	Splash, onboarding, login, OTP, profile, college selector, home, kitchens, food search, kitchen detail, item sheet, cart, checkout, confirmation, tracking, pickup QR, order history, receipt, support, profile/settings.
Kitchen	Login, registration, approval status, setup wizard, overview, live order queue, order detail, pickup verifier, menu, item editor, categories, inventory, business hours, staff, analytics, reports, settings.
Admin	Login/MFA, dashboard, colleges, college detail, kitchens/pending approvals, kitchen detail, students, live orders, transactions/refunds, promo codes, reports, support, audit logs, settings.

Primary student tabs: Home, Kitchens, Orders, Profile. Cart becomes a contextual sticky action, not a fifth permanent tab. Kitchen navigation favors high-contrast Orders as the most common action.

09 / Data model and integrity requirements

Domain tables	Important columns / relationships
Identity	users, sessions, user_roles, student_profiles, kitchen_staff; scope memberships and verified contacts.
Campus and vendor	colleges, kitchens, kitchen_hours, kitchen_closures, kitchen_documents, kitchen_pickup_counters.
Catalog	categories, menu_items, item_variants, addon_groups, addon_choices, item_images, inventory.
Commerce	carts (optional server-sync), orders, order_items, order_item_options, order_status_history, stock_reservations.
Payments	payment_attempts, payments, payment_webhook_events, refunds, financial_adjustments, payout_records (later).
Experience	favorites, notifications, notification_preferences, support_tickets, ratings, coupon_rules, coupon_redemptions.
Governance	audit_logs, system_settings, feature_flags, idempotency_records.

- UUID or bigint primary keys selected consistently; foreign keys required for tenant-critical relationships.
- Orders contain immutable snapshots of item name, variant/add-ons, tax and unit price in integer paise.
- Unique indexes on gateway events and idempotency keys to reject duplicates.
- All timestamps stored in UTC; display in Asia/Kolkata. Food opening hours represented in local campus time zone.
- Use order version column / locking to prevent concurrent invalid state changes.
- Never permanently delete financial history; archive or soft-disable relevant business entities.
- PII limited to operations need; clear retention/deletion policy before launch.

10 / Order state machine and monetary accounting

Keep order fulfillment status separate from payment status. For example, payment=PENDING/PAID/FAILED/REFUNDED/PARTIAL_REFUND and fulfillment=PENDING_ACCEPTANCE/ACCEPTED/PREPARING/READY/COMPLETED/REJECTED/CANCELLED. They are related but not interchangeable.

Transition	Actor	Preconditions / side effects
checkout -> pending	Student/backend	Validate scope, stock, current price, slot, idempotency; reserve inventory transactionally.
pending -> accepted	Kitchen staff	Valid paid/cash payment policy; active vendor; atomically update order version.
accepted -> preparing	Kitchen cook	Accepted only; record actor and time; notify student.
preparing -> ready	Kitchen cook	Pickup token generated or activated; notify student.
ready -> completed	Authorized cashier	Validate one-time QR/OTP, expiry, ownership; consume token atomically.
pending -> rejected	Kitchen	Require reason; release stock; if money captured initiate refund request.
active -> cancelled	Rules-based actor	Only permitted transitions; reason, release policy, fee/refund rules, audit.

Payment reconciliation and safety

- Never mark online payment paid from a client-side redirect alone; verify gateway evidence and webhook signature in live mode.
- Persist gateway events with unique event IDs and process each event idempotently.
- When gateway success and local timeout conflict, background reconciliation queries provider; display Processing rather than Failed.
- Every refund has initiated/pending/succeeded/failed states, source payment and operator reason.
- Financial reports reconcile paid, refunds, promotions, gateway fees, platform commission and vendor payout.

11 / API contract outline

Endpoint	Function / access
POST /v1/auth/otp/request, /verify	Identity; rate limits, short TTL, login session.
GET /v1/colleges, /v1/colleges/{id}/kitchens	Public approved discovery with server-side filters.
GET /v1/kitchens/{id}/menu	Catalog categories, variants, stock, accurate amounts.
POST /v1/kitchens/register	Owner applies; admin approval before visibility.
POST/PATCH /v1/kitchen/menu-items	Vendor-scoped authorized writes.
POST /v1/orders/quote	Authoritative price, fees, eligibility and slot revalidation.
POST /v1/orders	Idempotent authenticated order creation.
GET /v1/orders/{id}	Student owner, vendor staff, or authorized admin only.
POST /v1/kitchen/orders/{id}/accept|reject	Authorized vendor transition with expected order version.
PATCH /v1/kitchen/orders/{id}/status	Server validated status state machine.
POST /v1/kitchen/orders/{id}/verify-pickup	One-time claim, authorized pickup staff.
POST /v1/payments/webhook	Signed events, dedupe, fast acknowledgement.
GET /v1/admin/reports/...	Privileged aggregates with safe filtering.
WS /v1/realtime	Authenticated user/kitchen rooms; reconnect REST sync.

All list APIs require pagination; standard errors include code, message, field details, request_id. Preserve API version prefix. Provide OpenAPI schemas and examples for app integration.

12 / Background tasks, Redis and notifications

PostgreSQL remains the source of truth. Redis stores short-lived menu caches, rate-limit counters, ephemeral connected-session state and queued work. Use a reliable broker and job worker; Redis Pub/Sub delivery is best-effort, so clients always resync order state after reconnect.

- Jobs: send push events, generate receipts, reconcile payments, flag order acknowledgement timeout, release expired stock reservations.
- Keys: versioned and scoped by college/kitchen, e.g. kitchen:{id}:menu:v2, rate:otp:{hash}. Set explicit TTLs.
- Device notifications: permission-based push in production; while local, simulate an in-app inbox and visible event log.
- WebSocket: subscribe only after verifying membership/ownership; include order_id, type, version and timestamp.
- Outbox pattern recommended for durable event publication at scale; in MVP persist state + event record transactionally before dispatch.

13 / Local setup and developer workflow

Monorepo arrangement: apps/student (Expo), apps/kitchen (Expo), apps/admin (React web), backend/app (FastAPI modules), backend/tests, infra/docker-compose.yml, docs, shared design tokens and typed API client. Prefer two separate Expo applications to keep kitchen and student permissions and navigation separate.

Service	Local target / notes
Student Expo	Metro dev server, phone or Android emulator; web if compatible.
Kitchen Expo	Independent Expo app and dev server.
Admin web	http://localhost:3000 (configurable).
FastAPI	http://localhost:8000; interactive docs /docs.
PostgreSQL	localhost:5432, private local development credentials.
Redis	localhost:6379, not exposed publicly.
Worker	Processes notifications/reconciliation from queue.
Storage	Local file storage for development; cloud object storage adapter later.

Local environment sequence

- Install Node LTS, Python supported version, Docker Desktop/Engine, Git, and emulator or Android phone.
- Initialize monorepo and shared lint/format/type checks. Keep per-app .env.example committed, secrets .env ignored.
- Run Docker Compose for Postgres and Redis with health checks and named volumes.
- Run Alembic migrations and seed 3 colleges, 9 kitchens, ~180 products, staff, student accounts, and 200 test orders.
- Start FastAPI + worker; verify /health and /docs endpoints.
- Start Student and Kitchen Expo apps; connect API with environment-specific base URL. On physical phone use machine LAN IP rather than phone localhost.
- Start Admin panel; run end-to-end test from menu creation to pickup claim; restart containers to confirm persistence.

14 / Security, compliance and privacy

- JWT/session rotation and revocation; do not store refresh tokens in insecure plain client storage. SecureStore/Keychain equivalents on mobile.
- Enforce authorization on each read/write. Kitchen ID and college ID in request payload never grant permissions.
- OTP request/verify rate limiting and device abuse protection. Restrict login attempts and webhook origins.
- TLS for hosted production; local HTTP only for controlled development. Secrets in environment/secret manager, never source control.
- Payments: card/UPI information processed by compliant payment provider; never capture or store card credentials in CampusBite.
- Food vendors should supply appropriate registrations and campus permission as applicable; confirm legal and institutional requirements before launch.
- Avoid collecting precise live location after initial campus discovery unless essential and consented.

- Create privacy notice, data retention rules, account deletion workflow, support escalation and incident logging.
- Accessibility: screen reader labels, focus order, reduced-motion handling, sufficient color contrast, accessible pickup code alternative.

15 / Quality assurance and test matrix

Scenario	Expected result / pass condition
Two students buy last item	One succeeds; no oversell; failed transaction receives accurate message.
Checkout tapped twice	One order per idempotency key; no double debit.
Payment webhook repeated	Exactly once accounting effect; event duplicates ignored.
Kitchen offline/reconnect	New orders resync; latest state correct after restore.
Incorrect pickup token	Order stays ready; repeated attempts throttled.
Cross-college request	403/404 safe denial; no unauthorized user or vendor data.
Kitchen closes mid-order	Existing accepted order remains actionable with cancellation/refund options.
Price changed before checkout	New quote shown and reconfirmed; no silent increase.
Refund delayed	Transparent pending state, reconciliation and support path.
App under 100 concurrent sessions	No correctness regression; baseline latency and error rate collected.
500-1000 simulated sessions	Measure throughput, p95 response time, DB connections, CPU and memory; define budgets from measurements.
Accessibility / motion	All primary actions usable with reduced motion and text scaling.

Test levels and tools

Backend: pytest, HTTPX, transactional DB integration tests. Frontend: Jest/React Native Testing Library for UI and business state, Maestro or Detox for targeted flows. Load: Locust. Manual QA: low-end Android device, slow network, background/resume, notification permission denied, and offline recovery.

16 / Phased delivery roadmap and exit criteria

Phase	Build	Definition of done
0 - Foundation	Repository, ADRs, Docker, schemas, seed data, CI checks.	One-command local stack and documented smoke test.
1 - Design	Design tokens, component library, student and kitchen mock flows.	Screen flows reviewed; loading/error states and motion standards set.
2 - Core platform	Auth, colleges, vendor registration/approval, menu CRUD, roles.	Owner publishes approved products visible to correct college students.
3 - Commerce	Cart, quote, orders, stock reservation, kitchen queue.	Orders place reliably and appear in the right kitchen.
4 - Fulfillment	Realtime updates, status machine, ETA, pickup verification.	End-to-end student to verified pickup succeeds.
5 - Money + admin	Sandbox payments, refunds, basic reports, support.	Reconciled test transactions and role-safe administration.
6 - Hardening	Automated tests, performance, security and pilot feedback.	Critical scenario test suite passes; launch blocker list closed.

MVP non-goals

Do not initially build rider dispatch, GPS delivery tracking, cross-kitchen mixed-cart settlement, full meal subscriptions, AI forecasting, loyalty wallet, platform-wide dynamic pricing, or an elaborate microservices mesh. These can remain planned modules but should not delay correct counter-pickup operations.

17 / Operating metrics and launch KPIs

Metric	Formula / interpretation
Order conversion	Paid or approved orders / qualified checkout starts.
Acceptance time	Kitchen accepted_at - order submitted_at.
Prep lead time	ready_at - accepted_at; report p50, p90.
Pickup wait	completed_at - ready_at.
Cancellation rate	Cancelled/rejected orders / submitted orders, segmented by reason.
Repeat purchase rate	Students with >=2 completed orders within 30 days / active purchasers.
Vendor reliability	On-time readiness, acceptance %, refund/error rate.
Economics	GMV, platform commission, fees, refunds, vendor payout and contribution margin.

18 / Monetization and operational economics

Options include a vendor SaaS subscription, an order commission, or featured listings. Avoid assuming economics until a college pilot validates volumes, fee acceptance and payment costs. An illustrative scenario of 2,000 orders/day at INR 60 and 5% commission yields INR 6,000 gross daily commission; this is not net profit and excludes weekends, taxes, refunds, support and infrastructure costs.

- Keep commission rules versioned and attached to completed transactions.
- Choose payer of gateway fees clearly (vendor, platform or buyer) and disclose visible fees.
- Vendor payouts and split settlements require actual provider support and compliance review before enabling.
- Revenue dashboard must not label gross order value as business income.

19 / Edge cases and open product decisions

- What constitutes eligibility: student ID, college email, manual campus choice, or institution-approved invite?
- Can students at one college order from another college; default no.
- How should holidays, exam days, festivals and counter closures be reflected?
- Will pay-at-counter be offered, and what happens when a customer never collects?
- Who pays for cancelled paid orders and how are refund delays communicated?
- Is scheduled pickup mandatory for first pilot or introduced after immediate pickup?
- Should a kitchen accept orders without live stock counts, and how is unlimited stock represented?
- How will operator handle inappropriate food images, fake vendors, duplicate student accounts and disputes?
- Will the first release be Android-focused while web remains available for previews?
- What documents does each college require before kitchen onboarding?

20 / Definition of done: full MVP checklist

- Two React Native apps run locally; admin interface works in browser.
- Student registers, selects a campus and sees only approved kitchens.
- Owner registers a kitchen and admin approves it.
- Owner adds food images, prices, modifiers and open/closed status.
- Student adds food to cart, receives authoritative quote and submits order.
- Kitchen reliably receives order, accepts it, prepares it and marks it ready.
- Student receives real-time update and unique pickup token.
- Staff securely verifies pickup and finalizes order exactly once.
- Payment sandbox / cash policy, cancellations, stock restoration and refunds have tested paths.
- Admin can view authorized colleges, vendors, order history and summary reports.
- Backend role boundaries, financial idempotency and concurrency tests pass.
- Setup README, architecture map, API schema, seed accounts and test suite are documented.

21 / Suggested backlog: build order

Sprint A: repository, UI tokens, seed data, database migrations and role-scoped auth. Sprint B: student campus discovery and kitchen catalog; kitchen registration and menu CRUD; admin approvals. Sprint C: cart, server quote and idempotent order creation. Sprint D: kitchen queue, status transitions, realtime sync and pickup verification. Sprint E: sandbox payments, reconciliation, admin reporting and support. Sprint F: load, security, usability and pilot fixes. Tasks should be split vertically: backend route + UI + test + acceptance evidence, rather than only by technology layer.

22 / Sample end-to-end pilot walkthrough

01. Operator seeds North Campus and approves Campus Canteen.

02. Kitchen owner uploads Cheese Maggi at INR 60, 20 portions available, 12-minute prep estimate.

03. Student signs in, chooses North Campus manually, views the kitchen and adds two Maggi.

04. Checkout quotes INR 120 plus configured taxes/fees (if any), reserves stock and collects mock payment.

05. Kitchen receives order CB-1024, staff accepts, cook marks Preparing and then Ready.

06. Student order page updates without refresh; app displays code/QR with short expiration.

07. Cashier validates token, order becomes Completed; stock and analytics reflect sale.

08. Admin reviews college dashboard and correct transaction amounts; student can view receipt.

23 / Further expansion after successful college pilot

Prioritize what real usage shows: time-slot capacity management, canteen self-service kiosk mode, group orders, campus-specific offers, recurring mess plans, dietary preferences, multilingual UI, operations alerts, vendor staff shift assignments, smart demand forecasts, and optional on-campus delivery. Design extension points now without implementing speculative complexity too early.

24 / Implementation principles

- Make UX delightful but treat correct transactions and college isolation as first-order quality requirements.
- Start with a fast modular monolith, clear domain boundaries and reliable automated tests.
- Use original photography/assets and brand identity instead of replicating Blinkit screens pixel for pixel.
- Feature flags and realistic seeded environments make local demos reproducible.
- Separate measured launch requirements from aspirational roadmap features.
- Document each policy decision and revise this PRD when pilot evidence differs from assumptions.

END OF DOCUMENT